

AIQT Milestone 13 Build Specification v0.2

Planning-Phase Design System Skill

Field	Value
Product	AIQT CLI
Specification	Milestone 13 Build Specification
Version	v0.2
Milestone	M13 - Planning-Phase Design System Skill
Depends On	M1-M12 complete, including M12 Checkpoint and Graph Amendment Controls
Primary Agent Target	Claude Code, Codex, or equivalent coding/design agent
Implementation Status	Build-handoff candidate after v0.1 review corrections
Primary Design Principle	Improve planning guidance for UI work without adding commands or external execution

Document status: v0.2 resolves v0.1 review findings by adding behavioral workflow position, repository implementation scope, deterministic UI-heavy confidence scoring, skills-plan JSON integration, normalized validation commands, and required implementation report output.

1. Source Alignment

This document is derived from the AIQT MVP Specification v0.6, AIQT Technical Architecture Specification v0.3, and the completed milestone sequence M1 through M12. It assumes M12 is complete, merged to main, tagged, dogfooded, and that the repository contains the issue lifecycle, checkpoint amendment, dependency update, graph validation, and graph repair dry-run controls.

The governing design remains:

```
state -> graph -> packet -> checkpoint -> review/export -> recovery controls
```

M13 does not add a new workflow stage, command family, schema migration, design runtime, or external tool execution. It adds a planning-phase design-system guidance layer to improve the quality of UI-heavy plans and downstream agent packets.

2. v0.2 Change Summary

- Adds a Behavioral Position in the Workflow section to define exactly where M13 guidance is injected.
- Adds an explicit repository implementation scope and file map to avoid duplicated detection or guidance logic.
- Defines deterministic UI-heavy detection confidence scoring with strong technology and product signals.
- Clarifies that designAids is additive in aiqt skills plan JSON and must not replace M10 detectedIntegrations.
- Normalizes validation commands to pnpm scripts plus the established direct node_modules fallback.
- Adds a required implementation report / handoff summary matching M9-M12 practice.
- Clarifies that plan-critique-implement-validate is guidance emitted to the user-facing agent, not the build sequence for implementing M13.
- Preserves no-new-command scope and avoids design-system schema migration unless implementation proves unavoidable.

3. Milestone Objective

M13 adds a built-in planning-phase design-system skill. The goal is to make UI-heavy projects produce a coherent design-system baseline before implementation starts, so coding agents do not build generic, inconsistent UI work unit by work unit.

M13 must prove that AIQT can:

- detect UI-heavy projects deterministically from project context and repository signals;
- inject a design-system planning block into existing plan, driver, and interview prompts when appropriate;
- require or strongly recommend a Design System Foundation milestone/work unit during planning for UI-heavy projects;
- add design-system acceptance criteria to UI work-unit planning guidance;
- add bounded design guidance to aiqt next packets for UI work without expanding the work-unit scope;
- extend aiqt skills plan with advisory design aids while preserving M10 Supabase, Clerk, and shadcn/ui behavior;
- preserve all M1-M12 behavior and tests.

4. Product Decision

M13 is not a Claude Design integration, not a Hue integration, and not a design tool. It is a planning-guidance enhancement. The external coding/design agent remains responsible for design reasoning and implementation. AIQT supplies deterministic design-system instructions and acceptance criteria based on canonical project state.

Decision	Reason
No new commands	The design skill belongs inside the planning phase, not as another command surface.
No external execution	AIQT remains local/offline and does not call Claude Design, Hue, transitions.dev, or any remote service.
No automatic skill installation	M10 skills planning is advisory. M13 keeps that contract.
No design source of truth file	Design-system guidance is generated from canonical state and prompts; markdown/design docs are not canonical.
No schema migration by default	M13 should work through prompt templates and derived guidance unless implementation proves additive state is necessary.

5. Commands in Scope

M13 adds no new public commands. It modifies output or guidance for existing commands only:

- aiqt prompt plan
- aiqt prompt plan --json
- aiqt prompt driver
- aiqt prompt driver --json
- aiqt prompt interview
- aiqt prompt interview --json
- aiqt next
- aiqt next --json
- aiqt skills plan
- aiqt skills plan --json

M13 must not change the mutation contract for update, plan import, checkpoint, review, issue, or graph commands.

6. Out of Scope

- new commands such as aiqt design brief or aiqt design plan;
- Claude Design API calls or execution;
- Hue execution, brand scraping, URL analysis, or screenshot parsing;

- transitions.dev execution or motion tuning against a live app;
- automatic skill installation or package installation;
- new design-system schema in project.json/state.json unless unavoidable;
- generated UI code;
- automatic work graph mutation after planning;
- broad replanning;
- new checkpoint schema fields;
- package publishing or release packaging work.

7. Behavioral Position in the Workflow

M13 does not add a new workflow stage. It modifies guidance inside the existing flow:

Capture -> Plan -> Handoff -> Execute -> Checkpoint -> Review -> Continue

Workflow point	M13 effect
Capture / interview	aiqt prompt interview asks design-system discovery questions for UI-heavy projects.
Driver prompt	aiqt prompt driver tells the external agent to establish a design-system foundation before UI implementation.
Planning prompt	aiqt prompt plan injects the design-system planner and instructs the agent to include design-system foundation work.
Handoff packet	aiqt next adds bounded design guidance only when the selected work unit is UI-related.
Skills planning	aiqt skills plan adds advisory designAids while preserving existing integration detection.
Checkpoint/review/recovery	No schema or command behavior change in M13.

Packet render precedence is deterministic:

1. AIQT packet scope and selected work unit boundaries
2. Selected work unit content and acceptance criteria
3. Project constraints and non-goals
4. Design-system guidance, only when UI-heavy/UI-related
5. Relevant skills and design aids
6. Generic model knowledge

Design guidance must never expand the work unit. If the design-system guidance conflicts with packet scope, packet scope wins.

8. UI-Heavy Detection Contract

8.1 Signal categories

UI-heavy detection must be deterministic and shared by all M13 call sites. It must not be implemented separately in each command/template.

Signal type	Strong signals
Technology	Next.js, React, shadcn/ui, Tailwind, components.json, app/ or pages/ route structure, src/components/ui, package dependencies such as next, react, @radix-ui, tailwindcss, class-variance-authority.
Product	public website, marketplace, SaaS, dashboard, booking flow, onboarding flow, client/admin/professional roles, authenticated user journeys, profile pages, search/browse flows, forms.
Weak textual	generic mentions such as UI, frontend, page, screen, style, component without supporting technology or product signals.

8.2 Confidence scoring

The detector returns confidence = high, medium, low, or none. The implementation must follow this rule:

Confidence	Rule	Behavior
high	At least 2 strong technology signals and at least 1 strong product signal.	Inject full design-system planner into plan/driver/interview guidance; add design guidance to UI packets; include high-relevance design aids.
medium	At least 1 strong technology signal OR at least 2 strong product signals.	Inject design-system planner with recommendation language; add design guidance to clearly UI-related packets.
low	Only weak textual signals.	Mention design-system guidance as optional; do not force a design-system foundation work unit.
none	No meaningful UI/product signals.	Do not inject design-system guidance.

8.3 UI-related work-unit detection

For packet rendering, a selected work unit is UI-related when its title, objective, scope, acceptance criteria, suggested files, or agentContextRefs include strong UI terms such as page, route, screen, form, component, layout, responsive, accessibility, shadcn, Tailwind, dashboard, profile, onboarding, booking, search, public UI, admin UI, or files under app/, pages/, components/, src/components/, or similar repository conventions.

9. Built-in Design-System Planner Skill

M13 adds an internal planning skill/template, not an external Claude Skill installation. Suggested implementation name:

aiqt-design-system-planner

The planner should instruct the external agent to establish a design-system foundation before UI implementation starts. It should be injected into planning prompts for high/medium UI-heavy projects.

9.1 Required design-system dimensions

- product design intent and emotional tone;
- palette tokens and semantic color usage;
- typography direction;
- spacing scale;
- radius scale;
- shadow/elevation rules;
- layout grid and content width rules;
- component primitive usage rules;
- button, form, card, table/list, navigation, dialog, and feedback patterns;
- loading, empty, error, success, and disabled states;
- responsive breakpoints and mobile behavior;
- accessibility baseline: focus states, keyboard navigation, labels, contrast;
- motion principles: subtle, purposeful, no excessive animation;
- anti-patterns to avoid, including generic SaaS/card-grid/default-gradient UI unless justified.

9.2 External-agent process guidance

The planner may tell the user-facing coding/design agent to follow this process: plan the design direction, critique it, implement bounded UI, then validate. This is guidance that AIQT emits to the user-facing external agent. It is not the implementation sequence for the Claude Code agent building M13 itself.

10. Planning Prompt Requirements

10.1 Design System Foundation work

When UI-heavy confidence is high, aiqt prompt plan must instruct the agent that the resulting plan should include design-system foundation work before major UI implementation. This may be a milestone or one or more early work units depending on plan shape. If milestone generation is used, the title should be equivalent to Design System Foundation.

Example work unit to request from the planning agent:

```
{
  "clientKey": "design-system-foundation",
  "milestoneClientKey": "design-system",
  "title": "Design system foundation",
  "objective": "Define and implement the visual and component system used by all UI work.",
  "scope": [
    "Define product-specific visual direction and design intent",
    "Create Tailwind/shadcn-compatible tokens for color, typography, spacing, radius, and elevation",
    "Define reusable component usage rules for layout, forms, cards, lists, navigation, dialogs, and feedback states",
    "Implement one representative screen or shell that demonstrates the design system"
  ],
  "outOfScope": [
    "Do not implement all application pages in this work unit",
    "Do not introduce a second component library or visual system"
  ],
  "acceptanceCriteria": [
    "Design tokens are represented in code or configuration",
    "At least one representative UI surface uses the tokens",
    "UI avoids generic default SaaS styling unless explicitly justified",
    "Responsive and keyboard-accessible behavior is covered"
  ],
  "agentContextRefs": [],
  "suggestedFiles": ["tailwind.config.ts", "components.json", "src/components/ui", "src/app"],
  "validationCommands": ["pnpm validate"]
}
```

10.2 UI work-unit acceptance criteria

For UI-related work units in high/medium UI-heavy projects, the plan prompt must instruct the planning agent to add acceptance criteria similar to:

- Uses the established design tokens and component primitives.
- Reuses existing UI primitives instead of inventing local one-off styles.
- Covers loading, empty, error, success, and disabled states where applicable.
- Works at mobile and desktop breakpoints.
- Includes visible focus states and keyboard-accessible controls.
- Does not introduce a second visual system.
- Preserves usability and accessibility over decorative styling.

11. Prompt Driver and Interview Updates

11.1 aiqt prompt driver

When UI-heavy confidence is high/medium, the driver prompt must instruct the external agent not to jump directly into UI implementation. It should first use AIQT to capture enough product and design-system context, then create a plan that includes design-system foundation work before feature screens.

11.2 aiqt prompt interview

When UI-heavy confidence is high/medium or the rough idea implies a UI product, interview prompts should ask for:

- target users and roles;
- core user journeys and public/authenticated surfaces;

- brand or tone references if available;
- design-system preference, such as shadcn/ui + Tailwind;
- accessibility expectations;
- content density and layout preferences;
- mobile/responsive expectations;
- motion preference and restraint;
- UI anti-patterns the user wants to avoid.

12. Agent Packet Design Guidance

aiqt next packets may include a concise Design Guidance section when both conditions are true: the project is UI-heavy with high/medium confidence, and the selected work unit is UI-related.

Design Guidance

Design-system priority:

- Follow the project design-system foundation and existing tokens/components.
- Do not introduce a second visual system.
- Keep this work unit bounded to the selected packet scope.

UI quality constraints:

- Use product-specific visual direction rather than generic default SaaS styling.
- Cover responsive behavior and keyboard-accessible interaction.
- Include loading, empty, error, and success states where applicable.
- Preserve usability and accessibility over decoration.

Priority:

1. AIQT packet scope and out-of-scope
2. Current repository code and established design tokens
3. Project constraints and non-goals
4. Recommended design aids/skills
5. Generic model knowledge

The Design Guidance section must not appear in non-UI packets or low/none confidence projects unless the selected work unit is explicitly UI-related and the message is short/advisory.

13. Skills Plan Extension

M13 extends aiqt skills plan with advisory design aids. This is additive and must not replace M10 detectedIntegrations or notDetectedIntegrations.

13.1 JSON integration contract

```
{
  "detectedIntegrations": [],
  "notDetectedIntegrations": [],
  "designAids": [],
  "safetyNotes": []
}
```

The M10 integration IDs and behavior for Supabase, Clerk, and shadcn-ui must remain unchanged. designAids is a sibling field.

13.2 Advisory design aids

ID	Phase	Recommended use
frontend-design	before-ui-implementation	Use to define visual direction, tokens, layout, and design critique before coding.
design-critique	after-mockup-or-screenshot	Use to critique usability, visual hierarchy, consistency, accessibility, and priority fixes.
web-design-guidelines	after-ui-code-exists	Use to audit actual UI files against interface design guidelines and produce concrete fixes.

ID	Phase	Recommended use
hue	brand-reference-available	Use when a brand URL or screenshot exists and the agent needs to infer a coherent design system.
transitions-refine	after-ui-runs-locally	Use when motion/transitions need refinement against a running app.

All design aids are advisory. `installAutomatically` must be false. M13 must not execute installs, network calls, brand scraping, screenshot parsing, or browser-based refinement.

14. Data and Schema Contracts

M13 is expected to require no canonical schema change and no `AIQT_SCHEMA_VERSION` bump. Derived detection results and design guidance should be generated at runtime from `project.json`, `state.json`, `package.json`, repository paths, and existing service outputs.

If implementation proves that a schema field is unavoidable, it must be additive and optional, and the implementation report must justify why prompt-only derivation was insufficient. The default expectation is no schema change.

15. Canonical Mutation Rules

- prompt commands remain read-only unless existing `--out` behavior is already supported by prior milestones.
- `aiqt next` keeps its existing state mutation behavior when selecting a work unit; M13 only changes packet content.
- `aiqt skills plan` remains read-only and must not append runlog events.
- M13 must not mutate `project.json`, `state.json`, or `runlog.jsonl` for design guidance alone.
- M13 must not create `AIQT.md`, `AGENTS.md`, `CLAUDE.md`, `docs/`, `.milestones/`, `.tasks/`, or design-system markdown source files.

16. Runlog Rules

M13 introduces no new runlog event types. Existing commands keep existing runlog behavior. Prompt rendering, skills planning, and derived design guidance must not append runlog events. `aiqt next` continues to append whatever packet/workflow events were already defined by earlier milestones; M13 must not add additional design-specific events.

17. Error Handling and Exit Codes

M13 should not change exit-code behavior for any existing command. Existing command failures remain governed by their original milestones. Design-heavy detection must be best-effort and non-blocking.

Command area	Expected M13 exit-code impact
prompt plan/driver/interview	No new failure modes except existing invalid project/state/output-file behavior.
aiqt next	No new failure modes. Design guidance must not make packet generation fail.
aiqt skills plan	No new failure modes. Missing design signals yields empty or low-relevance designAids.
detection helper	Internal only; detector errors should degrade to low/none confidence unless canonical project/state is invalid by existing rules.

18. Repository Implementation Scope

The implementation may adjust file paths to match the actual repository structure, but it must keep detection and design guidance in shared helpers rather than duplicating logic across commands.

Likely new files:
`src/workflow/design/ui-heavy-detection.ts`
`src/workflow/design/design-system-planner.ts`
`src/workflow/design/design-guidance-rules.ts`
`src/templates/design-system-planner-template.ts`

```
default modified areas:
src/templates/plan-prompt-template.ts
src/templates/driver-prompt-template.ts
src/templates/interview-prompt-template.ts
src/services/agent-packet-service.ts
src/templates/agent-packet-template.ts
src/services/skills-detection-service.ts
src/services/skills-plan-template.ts
src/cli/commands/prompt-plan.command.ts
src/cli/commands/prompt-driver.command.ts
src/cli/commands/prompt-interview.command.ts
src/cli/commands/skills-plan.command.ts
```

```
tests:
tests/unit/ui-heavy-detection.test.ts
tests/unit/design-system-planner.test.ts
tests/integration/prompt-plan-design-system.test.ts
tests/integration/prompt-driver-design-system.test.ts
tests/integration/prompt-interview-design-system.test.ts
tests/integration/agent-packet-design-guidance.test.ts
tests/integration/skills-plan-design-aids.test.ts
```

Critical implementation rule: UI-heavy detection and design guidance must be shared helpers. Do not implement independent detection logic in prompt plan, prompt driver, prompt interview, packet rendering, and skills plan.

19. Runtime and Package Impact

- No new runtime dependency expected.
- No external API calls.
- No network access during AIQT command execution.
- No automatic installation of public skills.
- No AIQT_SCHEMA_VERSION bump expected.
- No package publishing or release pipeline changes.

20. Testing Scope

20.1 Unit tests

- UI-heavy detector returns high for Next.js + React/shadcn/Tailwind plus marketplace/dashboard/booking signals.
- UI-heavy detector returns medium for one strong tech signal or two strong product signals.
- UI-heavy detector returns low for weak textual UI mentions only.
- UI-heavy detector returns none for backend/CLI-only project context.
- Design-system planner produces required design dimensions and avoids new command/schema instructions.
- Design aid ordering is deterministic.

20.2 Integration tests

- aiqt prompt plan includes design-system planner and Design System Foundation guidance for high-confidence UI projects.
- aiqt prompt plan does not inject design-system requirements for CLI/backend projects.
- aiqt prompt driver instructs external agent to establish design system before UI work for UI-heavy projects.
- aiqt prompt interview asks design discovery questions for UI-heavy rough ideas.
- aiqt next adds Design Guidance section for UI-related work units only.
- aiqt next does not add Design Guidance section for non-UI work units.
- aiqt skills plan --json preserves detectedIntegrations and adds designAids as a sibling field.
- aiqt skills plan human output clearly marks design aids as advisory and non-installed.

20.3 Regression tests

- M10 skills plan Supabase, Clerk, and shadcn-ui detection remains unchanged.
- M8 stdin import behavior remains unchanged.
- M4/M10 relevant skills packet behavior remains compatible with Design Guidance render precedence.
- No new commands appear in CLI help unless existing help naturally reflects modified descriptions.
- No runlog events are appended by skills plan or prompt rendering.

21. Implementation Tasks

- Create shared UI-heavy detection helper with deterministic confidence scoring.
- Create design-system planner/guidance template.
- Inject design-system planner into prompt plan for high/medium UI-heavy projects.
- Update driver and interview prompts with design-system discovery and sequencing guidance.
- Add packet Design Guidance rendering for UI-related work units.
- Extend skills plan output with additive designAids.
- Add unit, integration, and regression tests.
- Preserve existing command contracts, runlog behavior, and schema version.

22. Validation Commands

Run the standard project validation scripts when available:

```
pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm validate
```

If pnpm validate remains blocked by the known ignored-build-scripts/esbuild gate in this environment, use the established direct-command fallback and report the caveat:

```
node_modules/.bin/tsc --noEmit
node_modules/.bin/eslint .
node_modules/.bin/vitest run
node_modules/.bin/tsc
```

23. Definition of Done

- No new public AIQT commands are added.
- No canonical schema migration or AIQT_SCHEMA_VERSION bump occurs unless explicitly justified.
- UI-heavy detection is deterministic, shared, and tested.
- Planning prompts for UI-heavy projects require or recommend design-system foundation work before feature UI work.
- UI work-unit planning guidance includes design-system acceptance criteria.
- aiqt next packets include bounded Design Guidance only for UI-related work units.
- aiqt skills plan includes additive designAids and preserves M10 integration behavior.
- All existing tests pass plus M13 tests.
- No generated docs/ or export artifacts are staged.
- Implementation report is returned using the Required Agent Output section.

24. Required Agent Output

After implementation, Claude Code must return a structured report containing:

- branch name;
- commit hash;
- files changed summary;
- source files created and modified;
- tests created and modified;
- whether schema changed;
- whether AIQT_SCHEMA_VERSION changed;
- whether commands changed;
- whether runlog events changed;
- validation results;
- final test count;
- deviations from this specification;
- deferred or unresolved items;
- working tree status.

25. Agent Execution Packet

You are implementing AIQT M13 - Planning-Phase Design System Skill.

Starting point:

- M1-M12 are complete on main.
- M13 adds no new public commands.
- M13 is a prompt/planning/packet/skills guidance enhancement.

Hard constraints:

- Do not add external API calls.
- Do not install skills automatically.
- Do not call Claude Design, Hue, transitions.dev, or any remote tool.
- Do not add generated markdown as source of truth.
- Do not mutate project/state/runlog for design guidance alone.
- Do not create a second workflow engine or duplicate detection logic across commands.

Implement:

1. Shared deterministic UI-heavy detection.
2. Built-in design-system planner guidance.
3. Prompt plan/driver/interview design-system injection for UI-heavy projects.
4. Bounded Design Guidance section in UI-related aiqt next packets.
5. Additive designAids in aiqt skills plan output.
6. Tests for all new behavior and regressions.

Return the required implementation report from Section 24.